Particle Physics Simulation

Debug report: defects, surprises and corrections from the build

Olajide Badejo

November 2026

Abstract

This is the companion to the main report and it ranks equally with it. The main report says what the engine does; this one says what went wrong on the way and what each failure taught, because in a project whose output is numbers the history of the numbers that were wrong is worth as much as the ones that were right.

Entries are grouped by theme. Environment covers the four toolchain defects that had to be cleared before a single kernel compiled, one of which would have silently produced a binary for the wrong GPU architecture. Performance covers the results that contradicted my expectations: fusion delivering about one percent or an unresolved difference where I expected ten to twenty, and shared memory tiling turning out not to be a bandwidth optimisation at all. Physics and tooling covers the defects in the correctness apparatus itself, including two cases where the test rather than the code was wrong, one of which passed a published gate on mismatched particles, and a data corruption bug that put a random seed into a tile width column.

The most valuable entries here are the ones where nothing crashed. A build targeting the wrong architecture, a benchmark reading a corrupted column and an optimisation that quietly does nothing all produce plausible output, and none of them announces itself.

Contents

1 Environment **2**
- 1.1 nvcc 13.3 refuses GCC 16 as a host compiler . . . 2
- 1.2 CUDA 13 removed clockRate and memoryClockRate . . . 2
- 1.3 CMake could not identify the CUDA compiler at all . . . 3
- 1.4 The build silently targeted the wrong GPU architecture . . . 3

2 Performance **5**
- 2.1 Kernel fusion, expected ten to twenty percent, measured about one . . . 5
- 2.2 Shared memory tiling is not a bandwidth optimisation on this problem . . . 6
- 2.3 The first default tile width is not the fastest . . . 6
- 2.4 Absolute timings depend on the thermal state of the card . . . 7

3 Physics and tooling **8**
- 3.1 The hash collision that would have counted neighbours twice . . . 8
- 3.2 A diagnostic that would have perturbed the run it measured . . . 9
- 3.3 The density gate that passed by measuring the wrong particles . . . 9
- 3.4 A test that failed for a reason that was entirely my fault . . . 9
- 3.5 A benchmark that recorded a random seed as a tile width . . . 10
- 3.6 LaTeX macro names cannot contain digits . . . 10
- 3.7 BibTeX put en dashes back after they had been removed . . . 11

4 What the failures had in common **12**
- 4.1 The dangerous defects were the silent ones . . . 12
- 4.2 A tolerance without a scale is not a tolerance . . . 12
- 4.3 Size the optimisation before writing it . . . 12
- 4.4 Verification has to be independent of the thing verified . . . 13
- 4.5 The environment is part of the experiment . . . 13

Chapter 1

Environment

Four defects stood between a clean checkout and a compiling kernel. None was interesting in itself; all four are recorded because three of them produce misleading rather than obvious failures.

1.1 nvcc 13.3 refuses GCC 16 as a host compiler

Symptom. Compiling a trivial CUDA file with `-ccbin g++-16`:

```
crt/host_config.h:137:2: error: #error -- unsupported GNU version! gcc
versions later than 15 are not supported!
```

Root cause. CUDA 13.3 supports GCC up to and including 15 and enforces that with a preprocessor check, so this is a deliberate refusal rather than a parse failure in some header. The machine has GCC 13, 14, 15 and 16 installed and defaults to 15, so the problem only appears if the first design's floor of GCC 16.1 is taken literally, which it was.

Options. Pass `-allow-unsupported-compiler` and use GCC 16 anyway; drop to GCC 15.2, the newest supported host compiler; or install an older toolkit.

Fix and why. GCC 15.2. What the floor was really buying is the C++23 host standard, and GCC 15.2 provides that in full. Suppressing the toolkit's own compatibility check on a project whose entire output is numbers that are supposed to be trustworthy is the wrong trade: a miscompiled force kernel that still runs is the worst failure mode available here. Dropping the toolkit would have given up first class `sm_120` support on this card.

Verification. A kernel plus a CUB radix sort compiles, links and runs correctly under `-ccbin g++-15 -arch=sm_120 -std=c++23`.

1.2 CUDA 13 removed clockRate and memoryClockRate

Symptom. The device probe would not compile:

```
error: class "cudaDeviceProp" has no member "clockRate"
error: class "cudaDeviceProp" has no member "memoryClockRate"
```

Root cause. Both fields were deprecated during the CUDA 12 series and removed in CUDA 13. The information is still available through `cudaDeviceGetAttribute`.

Fix and why. Query both through the attribute interface. This matters more than a compile fix normally would: the memory clock and bus width are what the theoretical peak bandwidth is computed from, and the probe reports that figure beside the measured bandwidth. Hard coding it from a data sheet would have meant quoting a number that had never been measured on the machine that produced the results, which is exactly what the no number without a run rule exists to prevent.

Verification. The probe reports 192 bits at the measured memory clock, giving 672.0 GB/s, which agrees with the manufacturer figure.

1.3 CMake could not identify the CUDA compiler at all

Symptom. Configuration failed inside `project()`, before any project code was reached:

```
/usr/include/c++/15/type_traits(3303): error: __bool_constant is not a template
Error limit reached.
100 errors detected in the compilation of "CMakeCUDACompilerId.cu".
```

The confusing part was that nvcc had just compiled my own test kernel against the same standard library without complaint.

Root cause. The difference was the standard flag. My test passed `-std=c++20` explicitly. nvcc 13.3 defaults to C++17, and the libstdc++ 15 headers it is paired with here use `if consteval` inside `bits/c++config.h`, which is C++23 syntax. Under the default standard the front end cannot parse its own standard library, so nvcc fails on an empty translation unit. Reduced to two lines the failure is unambiguous:

```
c++config.h(586): error: expected a "("
     if consteval { return true; } else { return false; }
```

CMake identifies the CUDA compiler by compiling a probe with `CMAKE_CUDA_FLAGS` and nothing else. `CMAKE_CUDA_STANDARD` has no effect there, because it applies to targets and no targets exist yet.

Fix and why. Seed `CMAKE_CUDA_FLAGS` with the standard before `project()`, guarded so a value passed on the command line still wins. Requiring every person who clones the repository to pass an extra flag would have broken the promise that a clean clone plus two make commands reproduces everything.

Verification. Configuration reports `The CUDA compiler identification is NVIDIA 13.3.73 with host compiler GNU 15.2.0`.

1.4 The build silently targeted the wrong GPU architecture

Symptom. None. Everything compiled and would have run. The only trace was a status line reading `CUDA architectures: 75`, which is Turing, not the Blackwell part in the machine.

Root cause. The default was being set with a guard placed after `project()`:

```
if(NOT DEFINED CMAKE_CUDA_ARCHITECTURES)
  set(CMAKE_CUDA_ARCHITECTURES 120)
endif()
```

Enabling the CUDA language inside `project()` assigns a default architecture immediately, so by the time the guard runs the variable is already defined and the branch never fires.

Fix and why. Move the guard above `project()` and make it a cache variable, which is the only position where it can observe the undefined state while still losing to a value passed on the command line.

The default is still `120` for development. `-DCPS_FAT_BINARY=ON` builds `physics_core` for six architectures, which is how the release and continuous integration builds are made.

Why this is the most important entry in the chapter. Nothing broke. The code would have run on the RTX 5070 through just in time compilation of the embedded PTX, correctly and at the wrong speed. Every timing number in the benchmark ladder would have been measured against a binary built for the wrong architecture, and the only symptom would have been results somewhat worse than expected, which is indistinguishable from the code simply being slow. A defect that produces plausible numbers is far more dangerous than one that crashes.

Verification. The status line now reads `CUDA architectures: 120`, and `cuobjdump --list-elf` on a built object reports `sm_120` SASS rather than PTX alone. The latter check is the real one: the status line reports intent, the object file reports fact.

Chapter 2

Performance

Two expectations in the original design were contradicted by measurement, and the first explanation of one of them was then contradicted as well. They are written down here because an expectation that was wrong and got corrected is more useful than one that was never recorded.

2.1 Kernel fusion, expected ten to twenty percent, measured about one

The expectation. Fusing the force evaluation with the Verlet update takes one launch instead of three and stops reading the acceleration array. Ten to twenty percent seemed about right for a memory bound kernel.

The measurement. Round by round, at the default tile width, fused over tiled is 0.9624 at 10,000 particles and 0.9905 at one hundred and fifty thousand, both beyond their interquartile ranges and the session drift of 0.19 percent. At 200,000 particles it is 0.9913 with a range of 0.0097, which does not resolve anything: the tiled trials are bimodal, near 43.2 and 43.6 milliseconds. I do not yet know why.

Root cause of the wrong expectation. Arithmetic intensity. The all pairs sum performs $O(N^2)$ arithmetic on $O(N)$ data, so the ratio of operations to bytes grows without limit with the problem size, and device memory is nearly idle: Nsight Compute puts it at 0.34 percent of peak for the naive kernel and 0.32 percent for the tiled one.

What I got wrong next. `v1.0.0` turned this into a ceiling. It called the saving a round trip of the acceleration array, $24N$ bytes written and read, priced that at the measured bandwidth, and said no implementation could beat the result. Three things were wrong with that. The fused kernel still wrote the array. The split path reads the array twice, once in each integrator launch, so the description of the traffic was wrong too; counted from the code, fusion removes $60N$ bytes, which is 0.0209 milliseconds or 0.048 percent of a step at 200,000 particles. And the measured gains are several times larger than any such figure, so it was never a ceiling. It prices one resource and says nothing about the rest of the step. The gain itself, 0.1 percent in `v1.0.0`, was a difference of 0.12 percent between separate blocks against a session spread of about 3 percent, so it could not be told from noise.

What I changed. The protocol, so a difference of a percent can be read. The source of the gain is still unmeasured. The fused kernel stays because it is faster where the difference is

resolved; `v1.0.0` also said it halves the memory footprint of a step, which is false, because it adds a second position buffer.

2.2 Shared memory tiling is not a bandwidth optimisation on this problem

The expectation. Tiling is conventionally introduced as a way of reducing device memory traffic, and I introduced it that way. The prediction that follows is that the naive kernel is limited by memory bandwidth and the tiled kernel relieves it.

The measurement. Tiling delivers a factor of 1.96, which is real and large. But device memory throughput is 0.34 percent of peak before the change and 0.32 percent after. Device memory was never the constraint, so the conventional explanation of why tiling helps is wrong here even though the tiling itself is right.

What is actually happening. The counters point at the first level cache and the load store pipeline:

	throughput, percent of peak				per launch	
variant	SM	L1 and load store	L2	device memory	load sectors	warp issue
naive	87.96	87.96	1.70	0.34	5,000.1M	54.98
tiled	82.84	45.50	1.75	0.32	78.3M	82.84
fused	83.49	45.85	1.71	0.35	78.3M	83.49

Global load sectors fall by a factor of 63.9 and load store instructions by a factor of 3.8. First level cache throughput falls from 88.0 percent of peak, close to saturation, to 45.5 percent. The decisive number is warp issue efficiency, which rises from 55.0 percent to 82.8 percent. Tiling moves the reads into shared memory and the load store pipe stops being the limit.

Why this mattered practically. Attacking the naive kernel's device memory traffic would not have addressed the unit the counters name as limiting it. The profile pointed at the right resource; reasoning from the familiar description would not have.

Control. The arithmetic instruction counts are equal across all three variants to within a fraction of a percent, which is what licenses attributing the difference to data movement rather than to the variants computing different things. Those same counters also let the conventional 20 floating point operations per interaction be checked against the 18.0 actually retired, counting a fused multiply add as two. Interactions per second is the measurement, and TFLOP/s depends on which count is used.

2.3 The first default tile width is not the fastest

Symptom. A sweep of tile widths at 100,000 particles shows a width of thirty two, the `v1.0.0` default, is well behind the widest of the four measured.

	tiled		fused	
tile width	ms per step	TFLOP/s	ms per step	TFLOP/s
32	12.144	16.47	12.133	16.48
64	11.830	16.91	11.787	16.97
128	11.626	17.20	11.497	17.40
256	11.514	17.37	11.388	17.56

Root cause. Two barriers are paid per tile, so a narrower tile pays them more often per unit of arithmetic. That fits the direction of the result, though the barrier cost was not measured on its own. A width of 256 runs at 11.514 milliseconds against 12.144, a difference of 5.2 percent, and a paired ratio of 0.9486 with an interquartile range of 0.0020.

Fix and why. `v1.0.0` kept thirty two and reported the sweep. The default is now 256, and the ladder was measured there, because the difference is far outside the session drift of 0.19 percent.

A defect in the sweep itself. The `v1.0.0` sweep timed the force kernel alone for tiled, with particles that never moved, against whole steps for fused, so any comparison across variants in its table was void. It also carried a claim here that thirty two was the slowest of the four, when its own table had sixty four slower. Both variants now time a whole step.

2.4 Absolute timings depend on the thermal state of the card

Symptom. In `v1.0.0`, a tile width sweep run immediately after the full benchmark sweep produced timings about fifteen percent higher than the same sweep run from cold. Nothing in the code had changed.

Root cause. Sustained clocks falling as the card heats is the likely reason, though I did not log clocks. Either way it is large enough to swamp several of the effects being measured here, including the entire fusion result.

First fix, and why it was not enough. `v1.0.0` compared within one session, kept the fastest of three trials, and claimed relative differences were stable to a tenth of a percentage point. They were not. Later measurement of the same binaries found them 4.7 percent apart in separate blocks and 1.5 percent apart the other way when interleaved, because drift is confounded with which variant ran when, and more trials do not help.

Fix and why. Measure in rounds: every configuration's first trial, then every configuration's second, with the order rotating. Report the median of ten trials with its dispersion. Take comparisons from the ratio inside each round, because two configurations in one round saw the same machine. Measure one reference configuration at the start, middle and end of the sweep, and treat the spread of those, 0.19 percent here, as the smallest difference worth quoting.

Verification. Rerunning the sweep on a later commit reproduced the tile ratio to within a tenth of a percent, and `scripts/check_provenance.py` holds every recorded row to a commit that resolves and a clean tree.

Chapter 3

Physics and tooling

The entries here are about the correctness apparatus rather than the physics kernels. Two of them are cases where the test was wrong, which is worth recording precisely because a wrong test is more dangerous than a wrong implementation: it either blocks correct work or, far worse, passes when it should not.

3.1 The hash collision that would have counted neighbours twice

The pathology. Spatial hashing is many to one by construction. Two distinct grid cells can map to the same bucket. The neighbour search visits the twenty seven cells around a particle and scans each one's bucket, so if two of those twenty seven cells collide in the table, the particles in that bucket are scanned twice and counted twice.

The reason this is nasty is that the usual defence does not work. Filtering candidates by distance removes particles from unrelated cells, because they are genuinely far away. It does nothing about double counting, because those particles really are within the smoothing radius; they are simply being added to the sum more than once. The density would come out high by an amount that depends on the hash function rather than on the fluid, and the fluid would still look like a fluid.

Fix and why. Each particle stores an exact cell identity packed into 64 bits as three 21 bit fields, and the bucket scan compares it before accepting a candidate. This rejects both classes at once, at the cost of one 64 bit compare per candidate. The alternative of deduplicating the twenty seven hashed buckets before scanning would cost hundreds of comparisons per particle rather than one per candidate.

Verification. This is the one defect in the project that was designed against rather than discovered, so the verification carries the whole weight. The density field produced by the hashed search is compared against a brute force $O(N^2)$ host sum over identical positions, and the worst relative difference across four thousand particles is 2.85×10^{-7}, about what single precision rounding gives. A single dropped or double counted neighbour would move a density by several percent. A separate health check counts particles sharing a bucket with a foreign cell and confirms the collision rate is real but small, so the test is exercising the defence rather than a table so sparse that no collisions occur.

3.2 A diagnostic that would have perturbed the run it measured

The problem. The fused variant carries the leapfrog half step velocity, which is not the velocity belonging to the current positions. The energy diagnostic needs the synchronised one. The obvious implementation is to shift the velocity array forward by half a step, take the measurement, and shift it back.

Why that is wrong. In single precision, adding a quantity and then subtracting it does not return the original bits. A run that happened to take a diagnostic would therefore follow a very slightly different trajectory from one that did not, which breaks the bit identity guarantee the resume path claims and makes the energy curve partly a measurement of its own observer.

Fix. The half step correction is applied inside the reduction kernel, which reads the velocity and the stored acceleration and forms $v + a\,\Delta t/2$ on the fly without writing anything back.

Verification. A test runs eighty steps taking a full diagnostic after every step, and asserts the final state is bit for bit identical to the same run with no diagnostics at all.

3.3 The density gate that passed by measuring the wrong particles

Symptom. None, which is the point. `v1.0.0` published an interior density error of 4.20 percent against a 5 percent limit, with an explanation about smoothing on a jittered lattice. It was found by review for this release rather than by any failure.

Root cause. Two defects, one hiding the other. The gate chose interior particles by position in lattice order and read densities in cell sorted order, so positions and densities belonged to different particles, and the figures were close to the statistics of the whole block. With the index fixed, the `v1.0.0` mass gave 7.61 percent, which fails that limit. And that mass was wrong: it assumed a spacing the lattice does not have, and ignored the kernel's own lattice sum error.

Consequence. The equation of state turns excess density into pressure, so the `v1.0.0` dam break interior started near 76 kPa against a hydrostatic 3.9 kPa. Maximum speed after 100 steps was 3.695 m/s. With the calibrated mass it is 0.625 m/s.

Fix. The index is fixed, the mass comes from a kernel sum over the lattice laid out, and new limits were committed before the code measuring them existed: the mean error is 0.03 percent, the RMS deviation 1.49 percent and the maximum deviation 6.34 percent.

Why it is in this report. The gate read well, passed, and came with a plausible story. A check that was never seen to fail for the right reason should not be trusted just because it passes.

3.4 A test that failed for a reason that was entirely my fault

Symptom. The SPH kernel unit test `KernelsAreContinuousAtTheSupportBoundary` failed on first run, while every other kernel test passed.

Root cause. The test asserted that each kernel evaluated just inside its support boundary was within 10^{-3} of zero, in absolute terms. These kernels carry a $1/h^6$ normalisation: at a smoothing radius of 0.1 the spiky gradient peaks around 1.4×10^5 and the viscosity Laplacian around 1.4×10^6. An absolute tolerance of 10^{-3} against quantities of that magnitude is not a statement about continuity, it is a statement about nothing. The implementation was correct throughout.

Fix. The test now compares each kernel at the boundary against its own peak value, which is what continuity actually means for a quantity with a scale, and separately asserts the kernels are exactly zero outside the support so the transition is genuinely continuous rather than merely small.

Lesson. A tolerance without a scale is not a tolerance. Every numerical gate in this project is now expressed relative to something: the magnitude of the quantity, the machine epsilon, or a bound derived from the arithmetic.

3.5 A benchmark that recorded a random seed as a tile width

Symptom. The tile width figure had four bars, one of which sat at a tile width of `20260802`. That number is the random seed.

Root cause. The tile sweep opened the output file twice. It created a truncating output stream to write the header, and then called the row appending helper, which opens the same path independently in append mode. The header stream stayed open and buffered for the duration of the sweep and flushed at offset zero when it was finally destroyed, overwriting the beginning of the file with the header text. What survived was a file whose first rows were partly overwritten, so the column boundaries shifted and fields were read from the wrong columns.

Fix. The header is written through the same append path as the rows, with the file removed first, so only one handle to the file exists at any time.

Why it is in this report. Nothing failed. The benchmark ran, the CSV parsed, the plot rendered, and the number was wrong. It was caught by looking at the figure and asking what a tile width of twenty million meant. The general form of this defect is that two independent handles to one file will corrupt it, and buffering makes the corruption appear far from its cause.

3.6 LaTeX macro names cannot contain digits

Symptom. The report failed to compile with `Missing \begin{document}` pointing at a line in the generated macro file that looked entirely reasonable.

Root cause. A LaTeX control sequence is made of letters only. The offending definition was

```
\newcommand{\Poly6NormalisationError}{...}
```

in which the name ends at the digit, so this defined a macro called `Poly` followed by the literal text `6NormalisationError`. Literal text in the preamble is what produces that error message, and it is reported at the point of the text rather than at the mistake.

Fix. The three offending macros were renamed to spell their digits, and the generator now rejects any macro name that is not purely alphabetic, so this class of mistake fails at generation time with a message that names the problem rather than at compile time with one that does not.

3.7 BibTeX put en dashes back after they had been removed

Symptom. The dash gate failed on the compiled PDF, reporting seven en dashes in the bibliography, after the source bibliography had been cleaned of double hyphens and the gate had reported the sources clean.

Root cause. The bibliography style is what dashifies page ranges. The `plain` style passes page fields through a function that converts any run of hyphens into a TeX en dash ligature when it writes the intermediate bibliography file. Cleaning the source had no effect because the source was never the problem.

Fix. Page ranges are written with the word `to`, which the style passes through untouched. This also makes the bibliography consistent with the prose, which is required to write ranges the same way.

Why this justifies the whole design of the gate. The rule forbids a character in the finished document, so the gate has to check the finished document. A source only check would have reported clean while every reference in the printed bibliography broke the rule. Extracting the text of the compiled PDF and scanning that is what caught it, and it is the reason the gate runs last in the report build rather than first.

Chapter 4

What the failures had in common

4.1 The dangerous defects were the silent ones

Of the 15 entries in this report, six are about plausible output that was wrong.

A density gate reading mismatched particles passes and comes with a story. A build targeting Turing instead of Blackwell compiles, links and runs. A benchmark whose columns have shifted parses cleanly and plots. An optimisation that relieves a resource nothing was waiting on delivers a small positive number that is easy to accept. A neighbour search that counts some particles twice produces a fluid that still behaves like a fluid. A diagnostic that perturbs the state it measures produces a smooth curve.

None of these announces itself, and none is found by testing that the program runs. Each was caught by a different mechanism, and the pattern is that each mechanism compares the thing against something independent: the density's index against the position's, the object file against the intended architecture, the figure against what a tile width could plausibly be, the optimisation against a paired measurement, the neighbour search against a brute force sum, the diagnostic run against an undiagnosed one.

4.2 A tolerance without a scale is not a tolerance

One defect came from this directly. The kernel continuity test compared a quantity of order 10^{6} against an absolute tolerance of 10^{-3} and failed a correct implementation. Had the comparison gone the other way, against a quantity of order 10^{-6}, it would have passed anything at all.

Every gate in the project is now expressed against a scale that means something. The momentum gate is bounded by $\epsilon_{\mathrm{f32}} \sqrt{N} n_{\mathrm{steps}}$, which is what single precision arithmetic can explain. It passes with six orders of magnitude to spare, which means a pass only rules out gross breakage, and `docs/physics_validation.md` now says so for each limit. The force gate is relative to the magnitude of the acceleration being checked. The density gate is relative to rest density.

4.3 Size the optimisation before writing it

The fusion result taught this lesson and then its limit. Pricing the bytes removed at the measured bandwidth takes two minutes and gives 0.048 percent of a step at 200,000 particles, which would have corrected an expectation of ten to twenty. But `v1.0.0` called such a figure an absolute ceiling, and the measured gains are several times larger. Such a sum bounds the cost of one resource, not the benefit of a change.

The same goes for tiling, which is worth a factor of 1.96 that no bandwidth argument can explain, because device memory was never the constraint.

4.4 Verification has to be independent of the thing verified

The strongest checks in this project are the ones where the reference was produced by a genuinely different route. The double precision host force implementation shares no code with the kernels and is written as literally as the force law can be written. The brute force density sum shares no code with the neighbour search. The chi squared test is checked by feeding it a deliberately wrong model that it must reject, so a test that could never fail would itself fail.

Where sharing is unavoidable it is deliberate and in the safe direction: the smoothing kernels and the force law are compiled from one header into both the device kernels and the host tests, so the two cannot drift apart. That is sharing an implementation to guarantee agreement, not sharing an assumption to guarantee a pass.

4.5 The environment is part of the experiment

Four of the 15 entries were toolchain problems, and one of them would have invalidated every performance number in the main report while leaving no trace except numbers that were slightly disappointing. The first phase of this build existed to probe the environment and record what it found, and that turned out to be time well spent rather than ceremony.